САМОСТОЯТЕЛЬНАЯ РАБОТА СТУДЕНТА

Дисциплина: Проектирование и применение NoSQL-технологий

Вариант 4. Заказы интернет-магазина

Технологии: MongoDB, Python, Visual Studio Code

Выполнил: _____

Группа: _____

Проверил: _____

Алматы — 2026

1. Цель проекта и предметная область

Разработать работающий учебный прототип учёта заказов интернет-магазина в MongoDB. Каждый заказ содержит встроенный массив позиций (embedding), фиксированную цену на момент покупки (snapshot) и сумму. Реализовать CRUD, восемь запросов, агрегацию выручки по категориям и автоматические тесты.

Сущности

Клиент (customer_id), заказ (_id, created_at, status, total_kzt), позиции заказа (product_id, наименование, категория, зафиксированная цена, количество и стоимость строки). Заказ выступает агрегатом, позиции включены непосредственно в документ.

Бизнес-правило

Для расчётов выручки учитываются только оплаченные, отправленные и доставленные заказы: paid, shipped, delivered. Незавершённые и отменённые заказы в выручку не входят. Это правило учебной модели.

Стек

Python 3, PyMongo 4, MongoDB Server, MongoDB Compass для проверки содержимого коллекции, Visual Studio Code для разработки и запуска.

2. Функциональные и нефункциональные требования

Функциональные требования

F1. Создать заказ и встроить в него одну или несколько товарных позиций.

F2. Сохранить snapshot цены, названия и категории каждой позиции.

F3. Получить заказ по идентификатору.

F4. Найти заказы клиента с сортировкой по дате.

F5. Найти заказы по статусу и временному интервалу.

F6. Обновить статус заказа.

F7. Удалить тестовый заказ.

F8. Рассчитать выручку по категориям и продажи товаров.

Нефункциональные требования

N1. Воспроизводимость: один скрипт создаёт 36 синтетических заказов.

N2. Корректность: валидировать количество товара и неотрицательность цены; автоматические тесты.

N3. Безопасность: не хранить реальные персональные данные и секреты в репозитории.

N4. Поддерживаемость: разделение доменной логики, запросов, доступа к БД и CLI.

3. Карта Access Patterns (8 запросов)

Q1 — Заказы клиента. Вход: customer_id. Ожидаемый результат: Список новых к старым. Приоритет: высокий.

Q2 — Детали заказа. Вход: _id. Ожидаемый результат: Один документ. Приоритет: высокий.

Q3 — Заказы по статусу. Вход: status. Ожидаемый результат: Список по дате. Приоритет: средний.

Q4 — Заказы за период. Вход: created_at range. Ожидаемый результат: Список по дате. Приоритет: средний.

Q5 — Расходы клиента. Вход: customer_id, status. Ожидаемый результат: Сумма оплаченных. Приоритет: средний.

Q6 — Выручка категорий. Вход: status, items[]. Ожидаемый результат: Суммы по категориям. Приоритет: высокий.

Q7 — Топ товаров. Вход: status, items[]. Ожидаемый результат: Топ по количеству. Приоритет: средний.

Q8 — Выручка по дням. Вход: status, created_at. Ожидаемый результат: Сумма по дням. Приоритет: средний.

Два нетривиальных запроса

Q6 выполняет фильтрацию по состоянию заказа, разворачивание items через $unwind, группировку по category_snapshot через $group и сортировку по выручке. Q7 аналогично агрегирует количество проданных единиц товара, суммарную стоимость и формирует рейтинг.

4. Выбор базы данных и архитектура

Выбрана документная NoSQL СУБД MongoDB. Один заказ и его позиции удобно извлекать единым документом без JOIN. Для аналитики применяется aggregation pipeline. Python CLI обращается к модулю запросов и репозиторию, а затем к MongoDB.

Альтернативы

PostgreSQL: строгие связи и транзакции, но отдельные таблицы заказов и позиций с JOIN. Redis: эффективный кэш, но не выбран как основное хранилище истории заказов. MongoDB: естественная модель вложенных товарных позиций, чтение заказов одним документом.

ADR

Решение ADR-001: MongoDB с коллекцией orders; исторические параметры позиций копируются в заказ при покупке. Причина — быстрый доступ к карточке заказа и корректная история цен. Компромисс — денормализация; изменив цену в каталоге, нельзя автоматически менять старые заказы. Подробности — docs/ADR.md.

Архитектурная схема

VS Code/CLI → Python (domain, repository, queries) → MongoDB Server (база sro4_shop, коллекция orders). Отдельно MongoDB Compass служит визуальному контролю. Исходная диаграмма приложена как docs/architecture.svg.

5. Документ MongoDB и индексы

```
{
  "_id": "ORD-001", "customer_id": "C001",
  "created_at": ISODate("2026-10-01T08:00:00Z"),
  "status": "paid", "total_kzt": 12990,
  "items": [{"product_id": "PR01",
    "name_snapshot": "Наушники",
    "category_snapshot": "Электроника",
    "unit_price_kzt_snapshot": 12990,
    "quantity": 1, "line_total_kzt": 12990}]
}
```

Это иллюстрация структуры: реальные значения каждого документа вычисляются скриптом scripts/seed.py. Для денежных величин применяются целые тенге, чтобы избежать неточности float.

Индексы

customer_id + created_at DESC — история заказов клиента; status + created_at DESC — выборка по статусу; items.category_snapshot — возможная фильтрация по категории. Для масштабирования диапазонного поиска по датам может потребоваться индекс created_at.

Snapshot цен

В create_order цены и названия копируются в массив items. После изменения исходного словаря товара сохранённое значение unit_price_kzt_snapshot не изменяется; это проверяется тестом.

6. Реализация и порядок запуска

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m scripts.seed
python -m src.main demo
python -m unittest discover -s tests -v
```

Команды выполняются в терминале Visual Studio Code из корня папки проекта при работающем MongoDB Server. Для проверки состояния зайти в MongoDB Compass по mongodb://localhost:27017, открыть базу sro4_shop и коллекцию orders.

Файлы проекта

src/domain.py — формирование заказа и snapshot; src/db.py — соединение и индексы; src/repository.py — CRUD; src/queries.py — восемь запросов; src/main.py — интерфейс CLI; scripts/seed.py — повторяемая загрузка 36 документов; tests/ — проверки.

CRUD-демонстрация

```
python -m src.main create --id DEMO-1000
python -m src.main get --id DEMO-1000
python -m src.main update --id DEMO-1000 --status delivered
python -m src.main delete --id DEMO-1000
```

7. Агрегации и результаты для проверки

Выручка по категориям — Q6

```
db.orders.aggregate([
  {"$match": {"status": {"$in": ["paid", "shipped", "delivered"]}}},
  {"$unwind": "$items"},
  {"$group": {"_id": "$items.category_snapshot",
    "revenue_kzt": {"$sum": "$items.line_total_kzt"}}},
  {"$sort": {"revenue_kzt": -1}}
])
```

Порядок: исключить новые и отменённые заказы, развернуть массив позиций в поток строк, сгруппировать суммы по зафиксированной категории, отсортировать по выручке. Фактические суммы получаются при локальном запуске и здесь не выдумываются.

Другие аналитические запросы

Q5 вычисляет суммы оплаченных заказов клиента. Q7 агрегирует продажи по товарам и выводит ТОП-5. Q8 суммирует суммы заказов по календарным дням UTC. Команды запуска: python -m src.main query --number 5 (или 6, 7, 8).

Демонстрационные скриншоты

Перед сдачей вставить реальные изображения терминала после seed, Q1–Q8, CRUD и MongoDB Compass. Скриншоты реальной работающей базы не включены, так как из среды подготовки нет доступа к локальному серверу студента.

8. Тестирование

Проверяемые сценарии

T1. Snapshot цены: цена в заказе сохраняется при последующем изменении исходного объекта товара. T2. Суммы строк и заказа: количество × цена равно line_total_kzt, а сумма позиций равна total_kzt. T3. Валидация: пустой заказ и нулевое количество отклоняются. T4. Генератор seed: создаёт 36 уникальных заказов, суммы корректны. T5–T7. CRUD: проверяются вызовы добавления/чтения, изменения и удаления через mock.

Результат проверки в среде подготовки

Команда python -m unittest discover -s tests -v: 7 tests, OK. Для этих unit-тестов не требуется запущенный сервер MongoDB. Интеграционное выполнение запросов к MongoDB необходимо проверить локально; в среде подготовки пакет PyMongo и сервер MongoDB отсутствуют.

Ожидаемый объём

После первичной загрузки seed в пустую базу должно быть 36 заказов. Повторный запуск использует upsert по тому же _id, не создавая дубликатов. Если в базе уже есть другие заказы, общее число документов может быть больше.

9. Ограничения, дальнейшее развитие и вывод

Ограничения

Прототип не подключён к платежной системе и не реализует пользователей, авторизацию, складские остатки, аудит изменения статусов, строгую допустимость переходов состояния. Индекс created_at нужно рассмотреть при большом объёме запросов за период. Для реальной выручки необходимы учёт возвратов, скидок и налогов; здесь используется учебная модель.

Вывод

Создан прототип на Python и MongoDB с моделью встроенных позиций и snapshot цен, 36 синтетическими заказами, реализацией CRUD, восемью запросами и набором автоматических тестов. Документная модель упрощает чтение заказа целиком, а aggregation pipeline обеспечивает анализ выручки по категориям, дням и товарам. Проверка серверной части должна быть выполнена локально с настоящей MongoDB.

Материалы для сдачи

Исходники и README — в корне проекта; архитектурная схема — docs/architecture.svg; ADR — docs/ADR.md; отчёт — docs/report.pdf. Презентация готовится отдельно. Репозиторий GitHub создаётся студентом самостоятельно.

Источники

Задание СРО «20 тем мини-проектов по лекциям №1–8», вариант 4; документация MongoDB по документной модели и aggregation; документация PyMongo по операциям CRUD.